ResearchMind: A Locally-Hosted, GPU-Aware Multimodal Retrieval-Augmented Generation Pipeline with Integrated Plagiarism Detection

Raghuvardhan Rao Molugu
Research Consultant
raghuvardhan@researchfoundation.in

***Abstract*—**We present ResearchMind, a research-assistant system that ingests heterogeneous source material—text, PDF and office documents, audio, video, and spreadsheets—and produces a structured synthesis using a locally hosted large language model (LLM), avoiding third-party cloud inference for the core summarization path. Extracted content is chunked, embedded, and indexed in a local vector store to support context-aware retrieval. We extend this pipeline with a Similarity Finder subsystem that fingerprints an uploaded paper, queries free and official academic search APIs (arXiv, Semantic Scholar, Crossref, and, where credentials are supplied, IEEE Xplore and the NASA Astrophysics Data System), and scores content overlap using a weighted combination of embedding-based semantic similarity and lexical n-gram overlap, flagging candidates above a configurable threshold as a chance of plagiarism. A central engineering contribution of this work is a GPU-compute-placement strategy that verifies device usability with a real inference call rather than trusting driver-level availability reporting; we document a concrete case in which an installed accelerator was reported as available by both the operating system and the machine-learning framework, yet failed at the kernel level due to a compiled-wheel/architecture mismatch, and describe the runtime fallback that keeps the system functional without operator intervention. The complete system is exposed through a dual-mode browser interface and was validated end-to-end against a live backend, a local LLM, and live external search APIs.

***Index Terms*—**retrieval-augmented generation, local large language models, multimodal document processing, plagiarism detection, GPU compute placement, vector embeddings, academic search APIs

I. Introduction

Researchers routinely work with source material spread across incompatible formats: PDF manuscripts, slide decks, recorded lectures, and tabular datasets. Synthesizing this material into a coherent overview, and separately verifying that a draft does not unintentionally overlap with existing published work, are both labor-intensive tasks that are increasingly delegated to AI systems. The dominant pattern for such systems is to route documents through a third-party cloud API, which raises privacy, cost, and availability concerns—particularly for unpublished or sensitive research material.

ResearchMind is designed around a different default: the ingestion, embedding, and summarization pipeline runs entirely on local infrastructure, using a locally hosted LLM server (Ollama), a local automatic speech recognition (ASR) engine, and a local embedding model backed by an on-disk vector store. The one capability that is inherently non-local—checking a manuscript against the wider published literature for content overlap—is implemented as an explicitly scoped, opt-in subsystem that reaches only a small set of academic search endpoints, each chosen for having an official, free access path rather than being scraped.

This paper makes four contributions. First, we describe a unified architecture for multimodal ingestion and retrieval-augmented synthesis that treats five distinct content modalities through a common downstream representation. Second, we describe a plagiarism/originality-checking subsystem, the Similarity Finder, built on a pluggable connector interface over official academic APIs, with a blended semantic–lexical scoring function. Third, and most novel from a systems perspective, we describe a GPU-compute-placement strategy that treats accelerator "availability" as something to be verified by executing real work, not merely queried from the driver, and we report a concrete hardware/software mismatch discovered during deployment together with the runtime mitigation. Fourth, we describe a browser-based interface that exposes both capabilities through a consistent design system, and report an end-to-end validation of the complete stack.

II. Related Work

Retrieval-augmented generation (RAG) conditions a generative language model on passages retrieved from an external corpus at inference time, improving factual grounding relative to parametric generation alone [2]. ResearchMind applies the same principle in a single-session setting: content extracted from a user's own uploaded files is embedded and indexed, and, when a research-focus prompt is supplied, semantically relevant chunks are retrieved and folded into the final synthesis prompt. The underlying LLM and embedding model are both built on the Transformer architecture [1]; the embedding model follows the Sentence-BERT approach of fine-tuning a Transformer encoder with a siamese/triplet objective so that cosine similarity between sentence embeddings is meaningful [3]. Approximate nearest-neighbor retrieval over

the resulting embeddings is performed with a Hierarchical Navigable Small World (HNSW) graph index [5], as implemented by the ChromaDB vector database used in this system. Speech and audio content is transcribed with a Transformer-based ASR model trained on large-scale weakly supervised data [4], executed locally through a CTranslate2-optimized inference engine. Content-overlap detection has traditionally relied on lexical fingerprinting (e.g., shingled n-gram matching); ResearchMind combines that lexical signal with dense semantic similarity so that paraphrased or conceptually overlapping —but not verbatim—text is also surfaced, at reduced confidence relative to near-exact matches.

III. System Architecture

Figure 1 shows the end-to-end summarization pipeline. An uploaded file set is routed by file extension into one of five categories—plain text, rich documents, audio, video, and spreadsheets—each handled by a dedicated processor. Text and document extraction uses format-specific libraries (PyMuPDF for PDF, python-docx and python-pptx for Office formats, odfpy for OpenDocument formats); audio and video are transcribed by a locally hosted Whisper model, with video first demultiplexed to a 16 kHz mono waveform; spreadsheets are summarized statistically (column typing, correlation analysis, categorical value counts) before being handed to the LLM for narrative interpretation. Independent files are processed concurrently through a thread pool, since the workload is I/O- and inference-bound rather than CPU-bound.

Every processor's output is chunked with a fixed character budget and overlap, embedded, and upserted into a persistent, cosine-indexed ChromaDB collection. A summarization stage merges all per-source outputs into a single budgeted context block; if the user supplies a research-focus prompt, the system additionally retrieves the most semantically relevant stored chunks and folds them into that context before a single call to the local LLM produces the final structured summary (topic, overview, key points, methodology, data insights, research gaps, and keywords).

The pipeline is exposed through a small REST surface: a single synthesis endpoint that accepts a multipart file set and an optional context string, a health endpoint that reports LLM reachability and the presence of each format-specific extraction dependency, and a model-listing endpoint used by operator tooling. Uploaded files are written to a per-request temporary directory that is removed once processing completes or fails, so no uploaded content persists on disk beyond the lifetime of a single request; only the derived vector-store index and search cache persist across requests, and both are reset or expired independently of the raw uploads.

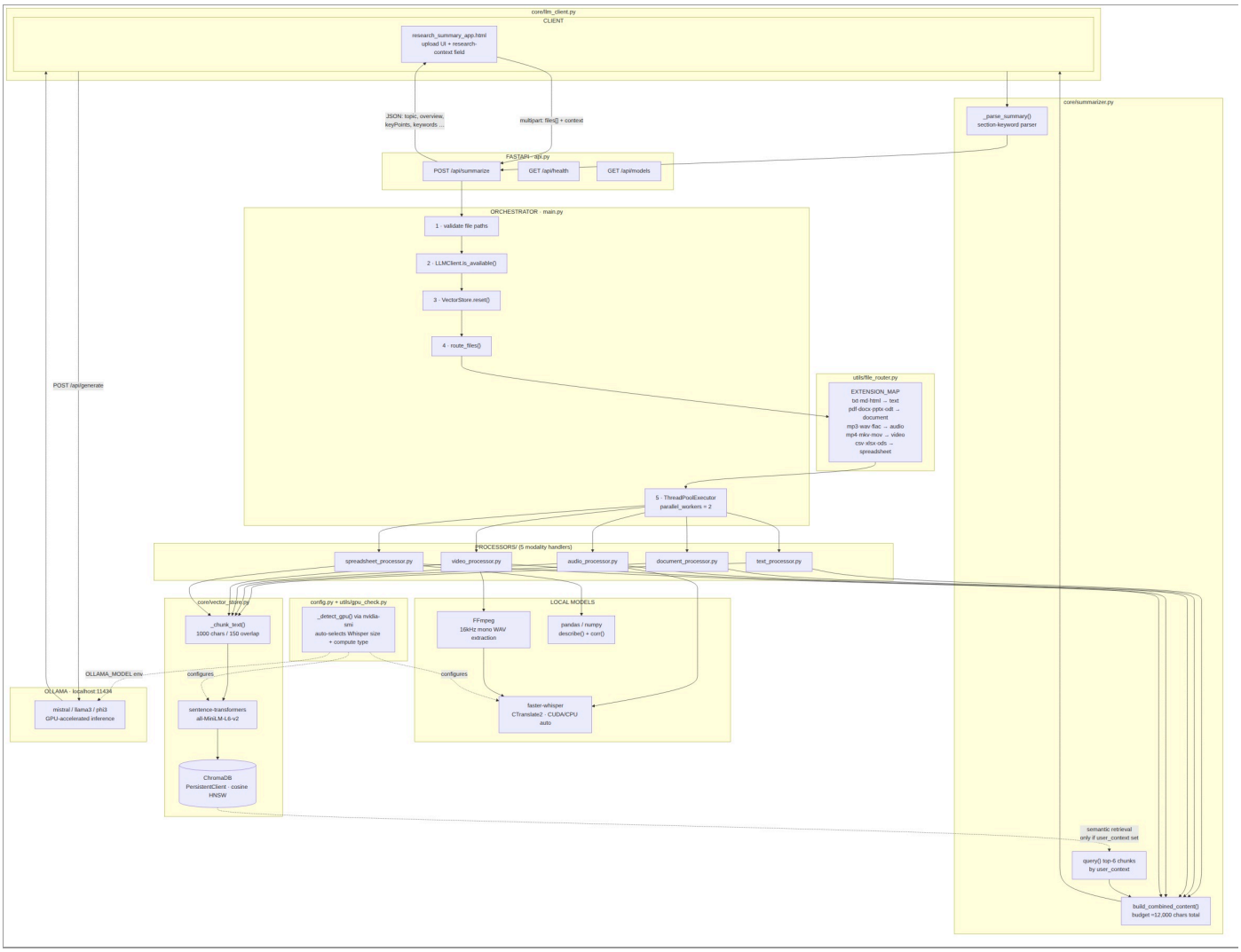

Fig. 1. End-to-end ingestion, retrieval, and synthesis pipeline. Files are routed by modality to dedicated processors, chunked and embedded into a local vector store, and merged into a single LLM synthesis call, with optional semantic retrieval when a research-focus prompt is supplied.

IV. Similarity Finder Subsystem

The Similarity Finder extends the pipeline with an originality check for a single uploaded paper (Fig. 2). It first fingerprints the document—extracting a title and abstract using the LLM where available, with a heading-and-paragraph heuristic fallback when the LLM is unreachable—and generates a small set of targeted search queries, again LLM-assisted with a keyword-extraction fallback. These queries are dispatched in parallel across a registry of search connectors, each implementing a common interface. Three connectors require no credentials: the arXiv API, the Semantic Scholar Academic Graph API, and the Crossref works API, which together give broad cross-publisher metadata coverage. Two further connectors are gated behind operator-supplied credentials: the IEEE Xplore API and the NASA Astrophysics Data System API, the latter chosen specifically because AAS-published journals (e.g., *The Astrophysical Journal*) are canonically indexed there. A connector that lacks its required credential is skipped automatically, with the gap reported through a status endpoint that the interface uses to grey out the corresponding option, rather than failing the request.

Deliberately absent are direct connectors to Google Scholar and the ACM Digital Library, neither of which exposes an official public search API; reaching them would require either a paid third-party proxy or scraping their result pages, the latter both fragile and contrary to those sites' terms of service. This gap is treated as a documented limitation rather than worked around silently. A separate, inert placeholder registry (covering PubMed, CORE, OpenAlex, DBLP, DOAJ, and several publisher APIs) defines the shape of additional sources that can be wired up later without modifying the fan-out or scoring logic.

Search results are deduplicated by DOI or normalized title, and content is resolved per candidate: where an open-access PDF is available (notably from arXiv) it is downloaded and

full text extracted through the same document processor used for uploads; otherwise the connector-supplied abstract is used, and each candidate carries a content-level flag reflecting which was obtained.

A. Blended Similarity Scoring

Both the uploaded paper and each candidate's resolved content are chunked with the same chunking function used elsewhere in the system, and pairwise cosine similarity is computed between all chunk embeddings. A semantic score is taken as the mean of the top-*k* highest-similarity chunk pairs, which is more robust to partial overlap (e.g., a shared methodology section in an otherwise distinct paper) than a single whole-document embedding comparison. A lexical score is computed independently as the Jaccard overlap between 5-gram word shingles of the full texts, which is more sensitive to near-verbatim copying than embedding similarity alone. The two are combined as

$$S = \alpha \cdot S_{sem} + \beta \cdot S_{lex}, \quad \alpha = 0.7, \beta = 0.3$$

Candidates with S at or above a configurable threshold (0.60 by default) are flagged as a chance of plagiarism; the same threshold and blended score drive both the API response and the interface's verdict banner, so the two never disagree. The report additionally distinguishes candidates resolved from full text against those resolved from an abstract only, since a high score against an abstract-only candidate reflects overlap in a much smaller sample of text and warrants more caution than the same score against a fully resolved source. Search results are cached on disk, keyed by a hash of the connector name and query string with a configurable time-to-live, so repeated checks against overlapping queries—common when a user revises and resubmits the same paper—do not repeatedly re-query rate-limited public endpoints.

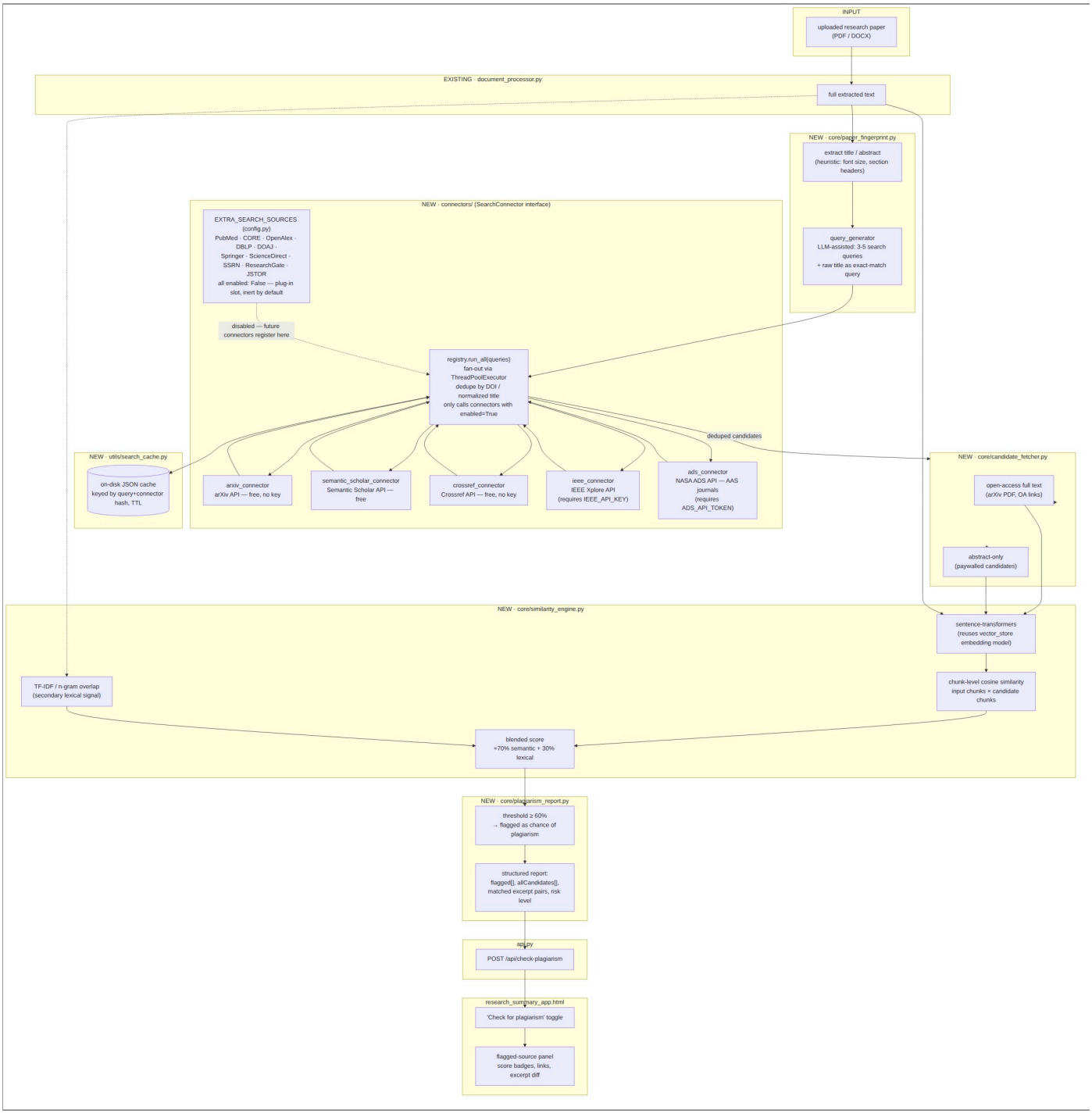

Fig. 2. Similarity Finder component architecture: fingerprinting, connector fan-out with on-disk caching, candidate content resolution, and blended-score reporting.

V. GPU-Aware Compute Placement

A recurring assumption in deployment guides for local-inference stacks is that querying the driver (e.g., `nvidia-smi`) or the machine-learning framework (e.g., `torch.cuda.is_available()`) is sufficient to decide whether GPU acceleration can be used. During deployment of this system we found that assumption to be false in a reproducible way, and we treat the resulting design as a contribution in its own right.

B. A Concrete Availability/Usability Mismatch

The deployment machine reports an NVIDIA GPU (8 GB VRAM, Pascal microarchitecture, compute capability 6.1) through both `nvidia-smi` and

`torch.cuda.is_available()`, the latter returning `True`. However, the installed PyTorch build—compiled against a newer CUDA toolkit—ships kernels only for compute capabilities 7.5 and above; it silently omits capability 6.1. A tensor placed on the device with `.to("cuda")` succeeds, since device placement is a memory operation, but the first operation that actually executes a kernel (e.g., an embedding lookup inside a forward pass) raises `no kernel image is available for execution on the device`. Framework-level availability checks and even successful model loading therefore give no guarantee that inference will succeed.

A second, independent issue compounded this: the embedding library used for the local vector store caches loaded models in a process-wide dictionary keyed only by model name, not by device. A first, failed attempt to load the embedding model on the GPU therefore poisoned the cache, so that a subsequent request for the same model on the CPU silently returned the already-instantiated, non-functional GPU-bound object—masking the fallback entirely rather than merely failing to apply it.

C. Verification-Before-Commitment Strategy

We address both issues with the same principle: before a device is committed to for the remainder of a process, it is exercised with a real, minimal unit of work, and only a caching-free path is used for that probe. For embeddings, a throwaway encoder instance (bypassing the vector-store library's cached wrapper) performs one real `encode()` call; if it raises, the system logs the failure and falls back to CPU before ever constructing the cached wrapper, so the poisoned-cache failure mode cannot occur. For speech transcription, the ASR engine reports, per device, which compute types it can execute; on the deployment GPU this excludes half-precision floating point even though enough VRAM is available to justify it by capacity alone. Requesting an unsupported compute type raises before any audio is processed, so the system catches that specific failure and retries with a supported compute type on the same device before considering CPU, keeping transcription GPU-accelerated whenever the device is usable at all. Table I summarizes the observed outcome for each component on the deployment hardware.

TABLE I
Requested vs. Actual Compute Placement (Deployment GPU: 8 GB, Compute Capability 6.1)

Component	Requested	Verified outcome	Reason
Sentence embeddings (vector store)	CUDA	CPU (fallback)	No kernel image for compute capability 6.1 in installed framework build
Sentence embeddings (similarity scoring)	CUDA	CPU (fallback)	Same root cause; verified independently via a real encode() call
Speech transcription (Whisper)	CUDA, half precision	CUDA, mixed 8-bit/32-bit	Inference engine reports half precision unsupported on this device; falls back to a supported compute type without leaving the GPU
LLM inference (Ollama, external process)	CUDA	CUDA	Managed independently by the LLM server; unaffected by the framework-level issue above

This design generalizes beyond the specific hardware observed: any deployment where the installed framework build and the physical accelerator diverge—a common situation as vendors phase out kernel support for older architectures in newer wheel releases—is handled the same way, without requiring the operator to pre-diagnose the mismatch.

VI. Web-Based User Interface

Both capabilities are exposed through a single-page browser interface (Fig. 3) with two modes sharing one visual design system. The Summarize mode accepts multiple files and an optional research-focus prompt and renders the structured synthesis described in Section III. The Similarity Finder mode accepts a single paper, optional title/author/domain-hint overrides, and per-source checkboxes that are populated at load time from the connector-availability endpoint, so credential-gated sources appear disabled with an explanatory label rather than silently failing later. The result view leads with a verdict banner—the highest blended score, a four-tier verdict label, and the titles of the top matching candidates—positioned above the general paper summary and the full candidate table, so the originality assessment is visible without scrolling. The four verdict tiers reuse the interface's existing accent palette rather than introducing new colors, keeping the two modes visually consistent. Because candidate titles, authors, and URLs originate from third-party APIs and are therefore untrusted input, they are rendered exclusively through safe DOM text-assignment rather than HTML interpolation, precluding script injection from a malicious or compromised upstream result.

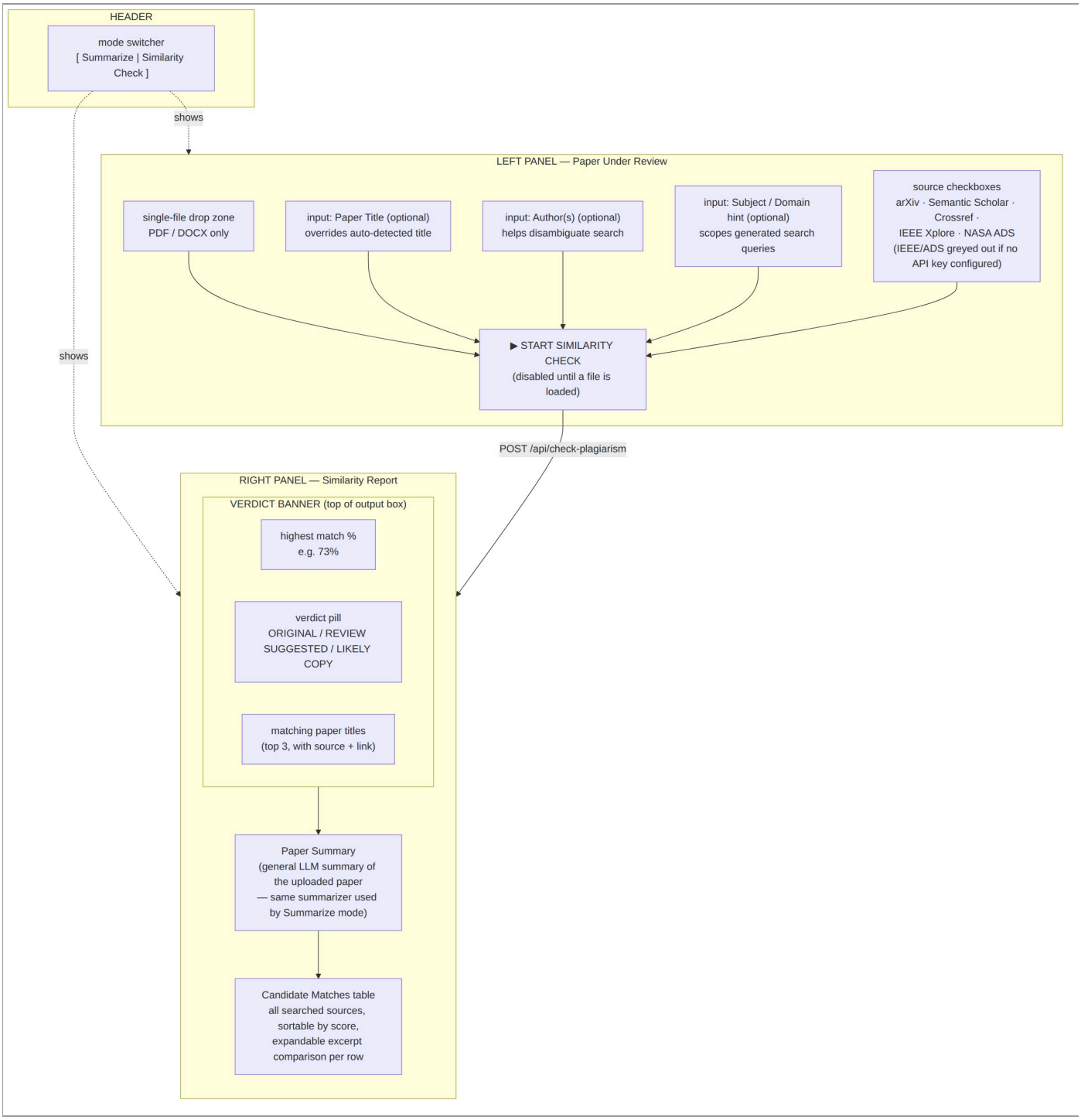

Fig. 3. Similarity Finder interface layout: input panel with per-source availability, and an output panel led by the verdict banner, followed by the paper summary and full candidate table.

VII. Empirical Observation

We validated the complete stack end-to-end with the back-end server, a local Ollama instance (7-billion-parameter instruction-tuned model), and live calls to the three keyless search connectors, driven through a headless-browser test of the actual interface rather than by calling internal functions directly. A short paraphrase of a well-known machine-learning paper was submitted to the Similarity Finder. The system returned 76 deduplicated candidates across the three connectors, with a highest blended similarity score of 48%, placing the result in the “low overlap, likely original” tier—below the 60% plagiarism-flag threshold. This is the expected outcome for a paraphrase that shares topic and terminology with the source

material without reproducing it verbatim, and indicates that the blended scoring function separates topical/conceptual overlap from the higher-confidence overlap that should be flagged. The same session exercised the Summarize mode against the identical input and produced a coherent structured summary, confirming both modes function correctly against the same underlying document-processing and LLM-integration code paths. No client-side errors were observed during either run.

VIII. Limitations and Future Work

- *Search coverage.* Google Scholar and the ACM Digital Library remain unreachable without either a paid proxy service or scraping; both are treated as an explicit gap rather than approximated.
- *Threshold calibration.* The 60% flag threshold and the four verdict-tier boundaries are engineering defaults, not calibrated against a labeled corpus of known-plagiarized and known-original submissions.
- *Single embedding model.* Semantic similarity relies on one general-purpose sentence-embedding model; domain-specific or multilingual papers may benefit from model ensembling or selection.
- *Reactive GPU verification.* The probe-then-fallback strategy in Section V detects a bad placement after attempting it, rather than consulting a precompiled compatibility matrix; this trades a small one-time verification cost for not requiring such a matrix to be maintained.
- *Additional sources.* Connectors for PubMed, OpenAlex, CORE, and several publisher APIs are scaffolded as configuration placeholders (Section IV) but not yet implemented, pending a per-source terms-of-service review.

IX. Conclusion

We described ResearchMind, a locally hosted multimodal research assistant that unifies document, audio, video, and spreadsheet ingestion behind a common retrieval-augmented synthesis pipeline, and extends it with a plagiarism/originality-checking subsystem built on official academic search APIs and a blended semantic–lexical scoring function. We further described and validated a GPU-compute-placement strategy that treats accelerator availability as something to be verified through execution rather than assumed from driver or framework reporting, motivated by a concrete, reproducible mismatch encountered on real deployment hardware, together with a related caching hazard in a third-party embedding library. The complete system, including both processing modes and the fallback behavior described here, was validated end-to-end through a live browser-driven test against the running backend, a local LLM, and live external APIs.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 5998–6008.

[2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2020.

[3] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2019, pp. 3982–3992.

[4] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," *arXiv preprint arXiv:2212.04356*, 2022.

[5] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, 2020.

[6] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North American Chapter Assoc. Comput. Linguistics (NAACL-HLT)*, 2019, pp. 4171–4186.

[7] Ollama Inc., "Ollama: Get up and running with large language models locally," 2024. [Online]. Available: https://ollama.com

[8] Chroma Inc., "Chroma: The open-source embedding database," 2024. [Online]. Available: https://www.trychroma.com